SYSTEM VERILOG INTERVIEW HANDBOOK

JAIRAJ MIRASHI
DESIGN VERIFICATION ENGINEER

Basic Level Questions

1. Difference between byte a and bit [7:0] a.

Both bit and byte are 2-state data types and can store 8-bit or 1-byte data. The difference between them is that 'bit' is unsigned whereas 'byte' is a signed integer.

2. What is the difference between bit and logic?

A bit is a 2-state data type having values as 0 or 1 whereas logic is 4 state data type having values as 0, 1, x, z.

3. Why logic is introduced in SV? Or Why reg and wires are not sufficient?

In Verilog behavior modeling, always, and initial procedural blocks use reg data type whereas, in dataflow modeling, continuous assignment uses wire data type. SystemVerilog allows driving signals in the 'assign' statements and procedural blocks using logic data type. The logic is a 4-state data type that allows capturing the Z or X behavior of the design.

4. Difference between reg and logic?

Both are 4 state variables. Reg can be only used in procedural assignments whereas logic can be used in both procedural and continuous assignments

5. What are 2 state and 4 state variables? Provide some examples.

2 state data type can store two values 0 and 1 4 state data type can store two values 0, 1, X, and Z

int	2 state data type, 32-bit signed integer
integer	4 state data type, 32-bit signed integer
shortint	2 state data type, 16-bit signed integer
longint	2 state data type, 64-bit signed integer

bit	2 state data type, unsigned, user-defined vector size
byte	2 state data type, 8-bit signed integer or ASCII character
logic	4 state data type, unsigned, user-defined vector size
reg	4 state data type, unsigned, user-defined vector size
time	4 state data type, 64-bit unsigned integer

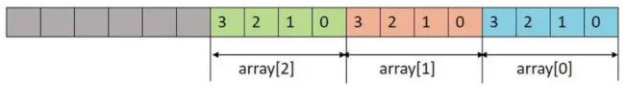
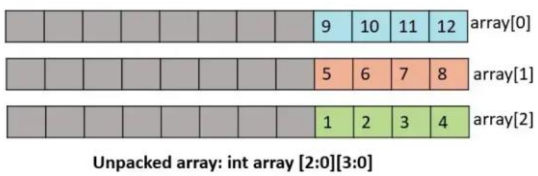
6. Difference between integer and int

Both can hold 32-bit signed integer values. The main difference between them is

integer – 4 state data type

int – 2 state data type

7. Difference between packed and unpacked arrays

Packed array	Unpacked array
A packed array refers to the dimension mentioned before the variable or object name. This is also known as the **vector width**. Memory allocation is always a continuous set of information that is accessed by an array index.	An unpacked array refers to the dimension mentioned after the variable or object name. Memory allocation may or may not be a continuous set of information.
`reg [5:0] array;  // [5:0] is packed dimension or vector width.` `bit [2:0][3:0] array;`	`reg arr [3:0];  // [3:0] is unpacked dimension.` `int array [2:0][3:0];`
 Packed array: bit [2:0][3:0] array	 Unpacked array: int array [2:0][3:0]
Example: `module array_example;`	Example: `module array_example;`

```
  bit [2:0][3:0] array = '{4'h2, 4'h4, 4'h6};
  initial begin
    foreach (array[i]) begin
      $display("array[%0h] = %0h", i, array[i]);
    end
  end
endmodule
```

Output:

```
array[2] = 2

array[1] = 4

array[0] = 6
```

```
  int array [2:0][3:0] = '{'{1, 2, 3, 4},
                           '{5, 6, 7, 8},
                           '{9, 10, 11, 12}
                          };
  initial begin
    foreach (array[i,j]) begin
      $display("array[%0d][%0d] = %0d", i, j, array[i][j]);
    end
  end
endmodule
```

Output:

```
array[2][3] = 1

array[2][2] = 2

array[2][1] = 3

array[2][0] = 4

array[1][3] = 5

array[1][2] = 6

array[1][1] = 7

array[1][0] = 8

array[0][3] = 9

array[0][2] = 10

array[0][1] = 11

array[0][0] = 12
```

8. Difference between dynamic and associative arrays

Dynamic Array	Associative array
Need to allocate memory before using it.	Memory can be allocated when it is used
Elements of an array have particular data types.	Elements of an array can be of any type. We can store the concatenation of various data types or class structures as well.

9. Difference between dynamic array and queue

Dynamic Array	Associative array
Memory needs to be allocated before using it i.e. array size is required to be allocated first.	For bounded queues, the size needs to be allocated. For unbounded queue can store unlimited entries.
Memory allocation is contiguous.	Memory allocation may not be contiguous and accessing intermediate variables needs to traverse like a linked list traversal.
new[ ] is used to create array memory.	Queue [<size>] for bounded queue size. Queue [$] for unbounded queue
Similar to the fixed array, any array element can be accessed.	Usually, head or tail elements are accessed. But queues also offer functionality to access any element of the queue.
The size can be increased using a new [ ] method to find out larger contiguous memory space and existing array elements get copied in new space.	For unbounded queues, the size of the queue expands using push_back or push_front methods. The new data elements get added similar to linked list node addition.

10. Difference between structure and union

Unions are similar to structures that can contain different data types members except they share the same memory location. Hence, it is a memory-efficient data structure. But it also restricts the user to use one member at a time.

11. How do you write a power b in sv code?

```
module arithmetic_op;
  reg [1:0] a, b;
  reg [3:0]out;
  initial begin
    a = 2'd3;
    b = 2'd2;
    out = a**b;

    $display("a = %0d and b = %0d", a, b);
    $display("pow: out = %0d", out);
  end
endmodule
```

12. What are pass-by-value and pass-by-reference methods?

A pass-by-value argument passing mechanism does copy arguments locally and operates on those variables. Any changes in variables in the function will not be visible outside of the function.

```
function int fn_multiply(int a, b);
```

A pass-by-reference argument passing mechanism does not copy arguments locally but reference to the original arguments is passed. This also means that any change in values for the argument inside the subroutine will affect the original values of the variables,

```
function int fn_multiply(ref int a, b);
```

EXAMPLE

13. Why do we need randomization in SystemVerilog?

Need for Randomization

As per the increasing complexity of the design, there are high chances to have more bugs in the design when it is written for the first time. To verify DUT thoroughly, a verification engineer needs to provide many stimuli. There can be multiple cross combinations of variables in a real system. So, it is not possible practically to write directed cases to verify every possible combination. So, it is very much required to have randomization in the verification testbench.

Advantages of Randomization

1) It has the capability of finding hidden bugs with some random combination.
2) Constraint-based randomization provides possible random values instead of a complete random range.
3) It provides flexibility to have random values based on user-defined probability.
4) SystemVerilog randomization provides flexibility to disable randomization for a particular variable in a class as well as disable particular constraints based on the requirement.
5) It saves time and effort in verification instead of writing a test for every possible scenario.

14. Difference between module and program block?

Difference between program and module block

1) A program block can not instantiate a module block. On the opposite side, a module block can instantiate another module or program block.
2) A program block can not have an interface, user-defined primitives (UDP), always block or nested program.
3) The initial block inside the program block is scheduled in the reactive region whereas the initial blocks inside the module lock are scheduled in the active region.

15. How do program block avoid the race condition?

To understand it clearly, below two points need to be cleared.

1) As per System Verilog scheduling semantic, System Verilog events are scheduled in the following order.
   Active region → Non-blocking region → Reactive region.
2) The non-blocking assignments (RHS) are evaluated in the active region and updates their LHS in the NBA region (Non-blocking region).

When the design module assigns some value to a variable in the initial block and the testbench module tries to access the same variable (in the initial block) and perform some action, race around condition is expected to occur.

Example:

Design Code:

```
module dut_example (output bit [3:0] out);
  initial begin
    out <= 2;
  end
endmodule
```

Race around solution example using program block

To resolve this race around the condition, a program block is used that executes in the reactive region. By the time the reactive region is scheduled, the out variable value would have updated to 2 in the NBA region.

TB Code:

```
program tb_pro(input [3:0] out);
  initial begin
    if(out == 2)
      $display("Design assigned out is 2");
    else
      $display("Design assigned out = %0d", out);
  end
endprogram

module tb_mod_top;
  wire [3:0] out;

  dut_example DUT(out);
  tb_pro tb(out);
endmodule
```

```
Output:
Design assigned out is 2
```

16. Difference between === and == operators?

The output of "==" can be 1, 0, or X. The output would be 'x', if you compare two variables if one or both the variables have one or more bits as X.

The output of "===" can only be 0 or 1. It is used to compare 'x' or 'z', whereas ambiguous values can not be compared using the '==' operator.

out = (A == B) and out = (A === B) results in tabular format.

A	B	Using ==	Using ===
0	0	1	1
1	1	1	1
0/1	x/z	x	0
x	x	x	1
z	z	x	1
x	z	x	0

17. What are SystemVerilog interfaces and why are they introduced?

System Verilog provides an interface construct that simply contains a bundle of sets of signals to communicate with design and testbench components.

Why are they introduced?

In Verilog for the addition of new signals, it has to be manually changed everywhere that module has been instantiated. System Verilog made it easier to add new signals in the interface block for existing connections.

Advantages:

1) It has increased re-usability across the projects.
2) A set of signals can be easily shared across the components bypassing its handle.
3) It provides directional information (mod ports) and timing information (clocking blocks).
4) Interfaces can contain parameters, variables, functional coverage, assertions, tasks, and functions.
5) Interfaces can contain procedural initial and always blocks and continuous assign statements.

18. What is modport and clocking block?

Within an interface to declare port directions for signals modport is used

To specify synchronization scheme and timing requirements for an interface, a clocking block is used.

SystemVerilog Modport

Within an interface to declare port directions for signals modport is used. The modport also put some restrictions on interface access.

Syntax:

```
modport <name> ( input <port_list>, output <port_list>);
```

Advantage of modport

1) Modport put access restriction by specifying port directions that avoid driving of the same signal by design and testbench.
2) Directions can also be specified inside the module.
3) Modport provide input, inout, output, and ref as port directions
4) Multiple modports can be declared for different directions for monitor and driver.

Examples:

```
modport TB (output a,b, en, input out, ack);

modport RTL (input clk, reset, a,b, en, output out, ack);
```

SystemVerilog Clocking Block

To specify synchronization scheme and timing requirements for an interface, a clocking block is used. The testbench can have multiple clocking blocks but only one clocking block per clock. The clocking block can be declared in the program, module, or interface.

Syntax:

```
clocking <clocking_name> (<clocking event>);

  Default input <delay> output <delay>;

  <signals>

endclocking
```

Advantages of clocking block

1) It provides a group of signals that are synchronous with a particular clock in DUT and testbench
2) Provides a facility to specify timing requirements between clock and signals.

Example:

```
interface my_int (input bit clk);
        logic [7:0] data;
        logic enable;

        //Clocking Block

        clocking dut_clk @(posedge clk);
            default input #3ns output #2ns;
            input enable;
            output data;
        endclocking

        // From DUT perspective, 'data' is output and 'enable' is input
        modport DUT (output data, input enable, clk);

        clocking tb_clk @(negedge clk);
            default input #3ns output #2ns;
            output enable;
            input  data;
        endclocking

        // From TestBench perspective, 'data' is input and 'enable' is output
        modport TB (input data, clk, output enable);


    endinterface
```

19. Difference between initial and final block.

Initial Block	Final Block
The initial block executes at the start of a simulation at zero time units.	final block executes at the end of the simulation without any delays
Usage: Initial configuration set-up	Usage: To display statistical information about the simulation

20. What is cross-coverage?

Cross coverage

The cross-coverage allows having a cross product (i.e. cartesian product) between two or more variables or coverage points within the same covergroup. In simple words, cross-coverage is nothing but a set of cross-products of variables or coverage points.

Syntax:

```
<cross_coverage_label> : cross <coverpoint_1>, <coverpoint_2>,...,
<coverpoint_n>
```

Example:

```
bit [7:0] addr, data;
bit [3:0] valid;
bit en;

covergroup c_group @(posedge clk);
  cp1: coverpoint addr && en; // labeled as cp1
  cp2: coverpoint data;       // labeled as cp2
  cp1_X_cp2: cross cp1, cp2; // cross coverage between two expressions
  valid_X_cp2: cross valid, cp2; // cross coverage between variable and
expression
endgroup : c_group
```

21. Difference between code and functional coverage

Code Coverage

Code coverage deals with covering design code metrics. It tells how many lines of code have been exercised w.r.t. block, expression, FSM, signal toggling.

The code coverage is further divided as

1) Block coverage – To check how many lines of code have been covered.
2) Expression coverage – To check whether all combinations of inputs have been driven to cover expression completely.
3) FSM coverage – To check whether all state transitions are covered.
4) Toggle coverage – To check whether all bits in variables have changed their states.

Note:

1) Code coverage does not specify that the code behavior is correct or not. It is simply used to identify uncovered lines, expressions, state transitions, dead code, etc. in the design. Hence, it does not indicate design quality.

2) Verification engineers aim to achieve 100% code coverage.
3) There are industry tools available that show covered and missing code in code coverage.

Functional Coverage

Functional coverage deals with covering design functionality or feature metrics. It is a user-defined metric that tells about how much design specification or functionality has been exercised. The functional coverage can be classified into two types

1) Data intended coverage – To check the occurrence of data value combinations. Example: Writing different data patterns in a register.
2) Control intended coverage – To check the occurrence of sequences in the intended fashion.
   Example: Reading a register to retrieve reset values after releasing a system reset.

Note:

Since it is user-defined metrics, it is up to the verification engineer to consider all features because if some features are missed to add in functional coverage and remaining features are covered then functional coverage will show up 100% even though some cover points are missed to add.

22. Different types of code coverage.

Code coverage deals with covering design code metrics. It tells how many lines of code have been exercised w.r.t. block, expression, FSM, signal toggling.

The code coverage is further divided as

1) Block coverage – To check how many lines of code have been covered.
2) Expression coverage – To check whether all combinations of inputs have been driven to cover expression completely.
3) FSM coverage – To check whether all state transitions are covered.
4) Toggle coverage – To check whether all bits in variables have changed their states.

23. Write rand constraint on a 3 bit variable with distribution 60% for 0 to 5 and 40% for 6,7. Write coverpoint for the same.

```
class rand_class;
  rand bit [2:0] value;

  constraint value_c {value dist {[0:5]:= 60, [6:7] := 40}; }

 covergroup c_group;
    cp1: coverpoint value {bins b1= {[0:5]};
                           bins b2 = {[6:7]};
                          }
  endgroup
endclass
```

24. How many types of arrays are there? Explain

1) **Fixed-size array in SystemVerilog:** Array size is fixed throughout the simulation. Its value will be initialized with a '0' value.
   The fixed size array can be further classified as a single-dimensional, multidimensional array, packed, and unpacked array
2) **Dynamic array in SystemVerilog:** An array whose size can be changed during run time simulation, is called dynamic array.
3) **Associative array in SystemVerilog:** An associate array is used where the size of a collection is not known or data space is sparse.

Intermediate level questions

1. How to find indexes associated with associative array items?

An array manipulation method find_index can be used for the indices of an associative array. Example:

```
function void find_index_method();
  int idx_q[$], idx;
  int qsize;

  // Find all idx having element as RED color
  idx_q = tr_assoc_arr.find_index with (item.colors == RED);
  qsize = idx_q.size;
  $display("Number of indexes having color item 'RED' = %0d", qsize);

  for(int i = 0; i < qsize; i++) begin
    idx = idx_q.pop_front();
    $display("Element %0d for popped index = %0d: ",i+1, idx);
    tr_assoc_arr[idx].print();
  end

  //expression has multiple conditions
  idx_q = tr_assoc_arr.find_index with (item.data == 3 && item.colors <=
RED);
  qsize = idx_q.size;
  $display("\nNumber of indexes for data == 3 and color == RED is %0d",
qsize);

  for(int i = 0; i < qsize; i++) begin
    idx = idx_q.pop_front();
    $display("Element %0d for popped index = %0d: ",i+1, idx);
    tr_assoc_arr[idx].print();
  end
endfunction
```

2. Difference between fork-join, fork-join_any, and fork-join_none

In fork-join, all processes start simultaneously and join will wait for all processes to be completed.

In fork-join_any, all processes start simultaneously and join_any will wait for any one process to be completed.

In fork-join_none, all processes start simultaneously and join_none will not wait for any process to be completed.

So, we can say that fork-join and fork-join_any is blocked due to process execution time, whereas fork-join_none is not blocked due to any process.

Refer SystemVerilog processes for more details and examples.

3. Difference Between always_comb and always@(*)?

always_comb	always@(*)
always_comb is automatically triggered once at time zero but only after all procedural blocks (initial and always blocks) have been started.	always @ (*) will be triggered for any change in the sensitivity list.
always_comb is sensitive to changes within the contents of a function.	always @* is only sensitive to changes to the arguments of a function.
Time constructs or delays are not allowed	Time constructs or delays are not allowed

4. Difference between structure and class

Structure	Class
A structure can contain different members of different data types.	Classes allow objects to create and delete dynamically.
Does not supports inheritance, polymorphism	Supports inheritance, polymorphism
Data members of structure are visible to everyone	Data members of class can be protected and will not be visible outside of class.
Supports data abstraction	Supports only grouping of data.

5. Difference between static and automatic functions

1) By default, functions declared are static except they are declared inside a class scope. If the function is declared within class scope, they behave as an automatic function by default unless they are specifically mentioned as static functions.
2) All variables declared in a static function are static variables unless they are specifically mentioned as an automatic variable.
3) All variables declared in an automatic function are automatic variables unless they are specifically mentioned as a static variable.

6. Difference between new[ ] and new()

new[ ] – To create a memory. It can also be used to resize or copy a dynamic array.

Example:

```
int array [];

array = new[5]; // create an array of size = 5

array = new[8] (array); // Resizing of an array and copy old array content
```

new() – To create an object for the class, commonly known as 'class constructor'.

```
class transaction;

  // class properties and methods

endclass

transaction tr; // variable of class data_type transaction or class handle

tr = new(); // memory is allotted for a variable or object.
```

7. Difference between shallow and deep copy

Shallow Copy:

The shallow copy is used to copy

1) Class properties like integers, strings, instance handle, etc
2) Nested objects are not copied, only instance handles are copied which means any changes are done in 'nested copied object' will also reflect in the 'nested original object' or vice-versa.

Deep Copy:

The deep copy is the same as shallow copy except nested created objects are also copied by writing a custom method. Unlike shallow copy, full or deep copy performs a complete copy of an object.

Refer for an example

8. How does the OOP concept add benefit in Verification?

Object Oriented Programming concept introduce concept of class and object in SystemVerilog similar to other programming language like C++, Java, Python, etc that provides following benefits

1) Inheritance: An Inheritance allows users to create an extended class from the existing class. This promotes code reuse and can lead to more efficient verification by having common functionality in the base class.
2) Polymorphism: Polymorphism means having many forms. A base class handle can invoke methods of its child class which has the same name. Hence, an object can take many forms.It uses virtual methods tha helps to override base class attributes and methods.
3) Data Encapsulation and Hiding: Data encapsulation is a mechanism that combines class properties and methods. Data hiding is a mechanism to hide class members within the class. They are not accessible outside of class scope. This avoids class member modification outside the class scope and its misuse. By default, all class members are accessible with class handles in SystemVerilog. To restrict access, access qualifiers are used.
4) Code Readability and Maintainability: The organized way of coding in OOPs provides readable and maintainable code which allows verification engineers to write tests and also build hierarchical verification testbench.

9. What is inheritance?

An Inheritance allows users to create an extended class from the existing class. The existing class is commonly known as base class or parent class and the newly created extended class is known as a derived class or child class or subclass. This promotes code reuse and can lead to more efficient verification by having common functionality in the base class.

Refer to an example:

```
class parent_trans;
  bit [31:0] data;

  function void disp_p();
    $display("Value of data = %0h", data);
  endfunction
endclass

class child_trans extends parent_trans;
  int id;
```

```
  function void disp_c();
    $display("Value of id = %0h", id);
  endfunction
endclass

module class_example;
  initial begin
    child_trans c_tr;
    c_tr = new();
    c_tr.data = 5; // child class is updating property of its base class
    c_tr.id = 1;

    c_tr.disp_p(); // child class is accessing method of its base class
    c_tr.disp_c();
  end
endmodule
```

10. What are the 'super' and 'this' keywords in SystemVerilog?

'super' keyword

The *'super'* keyword is used in a child or derived class to refer to class members of its immediate base class.

Refer to an example:

```
class parent_trans;
  bit [31:0] data;

  function void display();
     $display("Base: Value of data = %0h", data);
  endfunction
endclass

class child_trans extends parent_trans;
  bit [31:0] data;

  function void display();
    super.data = 3;
    super.display();
    $display("Child: Value of data = %0h", data);
  endfunction
endclass

module class_example;
  initial begin
    child_trans c_tr;
    c_tr = new();

    c_tr.data = 5;
    c_tr.display();
  end
endmodule
```

***'this'* keyword:**

To refer to class properties or methods of the current class instance, *this* keyword is used. In simple terms, *this* keyword is a handle of the current class object. It shall be used only in non-static class methods. The *'this'* keyword resolves the ambiguity of a compiler when class properties and arguments passed to class methods are the same.

Refer to an example:

```
class transaction;
  bit [31:0] data;
  int id;

  function new (bit [31:0] data, int id);
    data = data;
    id = id;
  endfunction
endclass

module class_example;
  initial begin
    transaction tr = new(5, 1);
    $display("Value of data = %0h, id = %0h", tr.data, tr.id);
  end
endmodule
```

11. What is polymorphism and its advantages?

Polymorphism means having many forms. A base class handle can invoke methods of its child class which has the same name. Hence, an object can take many forms.

Advantages

1) It makes code more reusable. The common functionality can be kept in the base class and derived class-based functionality will be alone in its inherited or child class. Thus it helps to write the code more modular.
2) Improves code readability and easy for code maintenance as common logic can be placed in the base class itself.
3) Provides an encapsulation that allows objects to expose only required functionality.
4) A new class can be added easily instead of modifying existing class functionality. Thus it enablesflexibility.

12. What is a virtual function?

A virtual function or task from the base class can be overridden by a method of its child class having the same signature (same method name and arguments).

In simple words, When a child class handle is assigned to its base class. On calling a method using a base class handle, the base class method will be executed. On declaring a method as a virtual method, a base class handle can call the method of its child class.

Usage:

1. Widely used in verification methodologies like UVM
2. Helps to reuse code
3. Helps to customize the behavior of verification components

Refer for more details and example:

```
class parent_trans;
  bit [31:0] data;
  int id;

  virtual function void display();
    $display("Base: Value of data = %0d and id = %0d", data, id);
  endfunction
endclass

class child_trans extends parent_trans;
  bit [31:0] data;
  int id;
  function void display();
    $display("Child: Value of data = %0d and id = %0d", data, id);
  endfunction
endclass

module class_example;
  initial begin
    parent_trans p_tr;
    child_trans c_tr;
    c_tr = new();

    p_tr = c_tr;
    c_tr.data = 10;
    c_tr.id = 2;

    p_tr.data = 5;
    p_tr.id = 1;
    p_tr.display();
  end
endmodule
```

13. What is the use of a scope resolution operator?

The scope resolution operator is used to

1) Access to static members (methods and class properties)
   Example:

```
class transaction;
  bit [31:0] data;
  static int id;

  static function disp(int id);
    $display("Value of id = %0h", id);
  endfunction

  function auto_disp(int id);
    $display("Value of id = %0h", id);
  endfunction
endclass

module class_example;
  initial begin
    transaction::id = 5;
    transaction::disp(transaction::id);

    //transaction::data = 2; // illegal
    //transaction::auto_disp(transaction::id); // illegal
  end
endmodule
```

2) Class method declaration outside the class.
   Example:

```
class transaction;
  bit [31:0] data;
  int id;

  extern function void display();
  extern task delay();
endclass

function void transaction::display();
  $display("data = %0d and id = %0d", data, id);
endfunction

task transaction::delay();
  #50;
  $display("Time = %0.0t, delayed data = %0d", $time, data);
endtask

module class_example;
  transaction tr;

  initial begin
    tr = new();
```

```
    tr.data = 100;
    tr.id = 1;

    tr.display();
    tr.delay();
  end
endmodule
```

3) Importing a package and also accessing package items. Example:

```
package pkg;
  class transaction;
    int data = 5;

    function void display();
      $display("data = %0d", data);
    endfunction
  endclass

  function pkg_funct();
    $display("Inside pkg_funct");
  endfunction
endpackage

//-------------------------------

import pkg::*;
module package_example;
  initial begin
    transaction tr = new();
    tr.display();
    pkg_funct();
  end
endmodule
```

14. Difference between virtual and pure virtual function

Virtual function	Pure virtual function
A virtual function from the base class can be overridden by a method of its child class having the same signature (same function name and arguments).	A pure virtual function is a method that makes it mandatory for methods to be implemented in derived classes whose prototypes have been specified in an abstract class.

<table>
<tr>
<td>If there is no implementation in the derived class, the base class's implementation is executed.</td>
<td>If a derived class doesn't overwrite the pure virtual function, it will remain abstract and cannot be
instantiated.</td>
</tr>
<tr>
<td>Declared by using the keyword 'virtual' in the base class.</td>
<td>Declared by using the keyword 'pure virtual' in the base class.</td>
</tr>
<tr>
<td>Example:

```
class parent_trans;
  bit [31:0] data;
  int id;

  virtual function void display();
    $display("Base: Value of data =
%0d and id = %0d", data, id);
  endfunction
endclass

class child_trans extends
parent_trans;
  bit [31:0] data;
  int id;
  function void display();
    $display("Child: Value of data
= %0d and id = %0d", data, id);
  endfunction
endclass

module class_example;
  initial begin
    parent_trans p_tr;
    child_trans c_tr;
    c_tr = new();

    p_tr = c_tr;
    c_tr.data = 10;
    c_tr.id = 2;

    p_tr.data = 5;
    p_tr.id = 1;
    p_tr.display();
  end
endmodule
```

</td>
<td>Example:

```
virtual class parent_trans;
  bit [31:0] data;
  int id;

  pure virtual function void
display();
endclass

class child_trans extends
parent_trans;
  function void display();
    $display("Child: Value of data =
%0h and id = %0h", data, id);
  endfunction
endclass

module class_example;
  initial begin
    parent_trans p_tr;
    child_trans c_tr;
    c_tr = new();

    p_tr = c_tr;
    p_tr.data = 5;
    p_tr.id = 1;
    p_tr.display();
  end
endmodule
```

</td>
</tr>
</table>

15. What is a virtual interface and its need?

An interface represents signals that are used to connect design modules or testbench to the DUT and is commonly known as a physical interface. The design and physical interface are static in nature. Hence, they can not be used dynamically. In modern testbench, randomized class objects are used and connect to the design dynamically. Hence, to bridge the gap between the static world of modules and the dynamic world of objects, a virtual interface is used as a pointer or handle for an actual interface.

16. What is a virtual class?

SystemVerilog Abstract class

An abstract class is a special type of base class that is not intended to be instantiated and a set of derived classes can be created.

1) An abstract class is an incomplete class that may contain method implementation or may contain only the prototype of methods without actual implementation (known as pure virtual methods). It can not be instantiated and it can only be derived.
2) The virtual keyword is used in front of the class to differentiate it from the normal class.
3) An abstract class is also known as a **virtual class**.
4) Method type, number of arguments, and return type (if required) should be the same for the virtual methods in their derived classes.
5) It is not mandatory to add methods in the abstract class.

Syntax:

```
virtual class <class_name>

  ...

endclass
```

Advantages of Abstract class

1) To keep the program organized and understandable, it forms a group of classes.
2) Common methods can be placed in the abstract class and these methods can be inherited by derived classes.

17. What are parameterized classes?

Parameterized classes are useful when the same class needs to be instantiated differently. The default parameter can be set in the class definition. These parameters can be overridden when it is instantiated.

Refer to an example:

```
class transaction #(parameter WIDTH = 2, type D_TYPE = bit [2:0]);
  bit [WIDTH-1:0] data;
  D_TYPE id;

  function void display();
    $display("data = %0d, id = %0d", data, id);
  endfunction
endclass

module class_example;
  transaction tr1;
  transaction #(3,int) tr2;

  initial begin
    tr1 = new();
    tr2 = new();

    tr1.data = 7;
    tr1.id = 15;
    tr1.display();

    tr2.data = 7;
    tr2.id = 15;
    tr2.display();
  end
endmodule
```

18. What are rand and randc methods?

rand Keyword

On randomizing an object, the rand keyword provides uniformly distributed random values.

```
rand bit [4:0] value;
```

On randomizing, any values within 5'h0 to 5'h1F will be generated with equal probability

randc Keyword

On randomizing an object, the randc keyword provides random value without repeating the same value unless a complete range is covered. Once all values are covered, the value will repeat. This ensures that to have all possible values without repeating the same value unless every value is covered.

```
randc bit [1:0] value;   // Possible values = 0, 1, 2, 3
```

Possible random value generated: 2, 3, 1, 0, 3, 2, 0, 1..

19. What are pre_randomize and post_randomize methods?

The *randomize()* method call is used to randomize class variables based on constraints if they are written. Along with *randomize()* method, SystemVerilog provides two callbacks

1. pre_randomize()
2. post_randomize()

A sequence of execution of methods:

```
pre_randomize() -> randomize() -> post_randomize()
```

pre_randomize method

It is used to do an activity just before randomization. This may involve disabling constraint for a particular variable (constraint_mode(0))or disabling randomization itself (rand_mode(0)). Refer disable randomization for more details.

post_randomize method

It is used to do an activity after randomization. This may involve printing randomized values of a class variable. Override the randomized value of a class variable.

pre_randomize and post_randomize methods Examples

A basic example

A basic example with pre_randomization and post_randomization methods which also include constraint_mode(0).

Example:

```
class seq_item;
  rand bit [7:0] val1;
  rand bit [7:0] val2;

  constraint val1_c {val1 > 100; val1 < 200;}
```

```
  constraint val2_c {val2 > 5; val2 < 8;}

  function void pre_randomize();
    $display("Inside pre_randomize");
    val2_c.constraint_mode(0);
  endfunction

  function void post_randomize();
    $display("Inside post_randomize");
    $display("val1 = %0d, val2 = %0d", this.val1, this.val2);
  endfunction

endclass

module constraint_example;
  seq_item item;

  initial begin
    item = new();
    item.randomize();
  end
endmodule
```

20. Explain bidirectional constraints

Bidirectional constraints are used to specify a relationship between two or more variables or signals where one variable value has a dependency on other variables.

Refer to an example,

```
class seq_item;
  rand bit [7:0] val1, val2, val3, val4;
  rand bit t1, t2;

  constraint val_c {val2 > val1;
                    val3 == val2 - val1;
                    val4 < val3;
                    val4 == val1/val3; }
  constraint t_c { (t1 == 1) -> t2 == 0;}
endclass

module constraint_example;
  seq_item item;

  initial begin
    item = new();

    repeat(5) begin
      item.randomize();
      $display("val1 = %0d, val2 = %0d, val3 = %0d, val4 = %0d", item.val1,
item.val2, item.val3, item.val4);
      $display("t1 = %0h, t2 = %0h", item.t1, item.t2);
    end
  end
endmodule
```

21. Is it possible to override existing constraints?

Yes, there are two ways to do so

1) Inline constraint:

```
class seq_item;
  rand bit [7:0] val1, val2;

  constraint val1_c {val1 > 100; val1 < 200;}
  constraint val2_c {val2 > 5; val2 < 80;}
endclass

module constraint_example;
  seq_item item;
  initial begin
    item = new();
    repeat(5) begin
      item.randomize();
      $display("Before inline constraint: val1 = %0d, val2 = %0d",
item.val1, item.val2);

      item.randomize with {val1 > 150; val1 < 160;};
      item.randomize with {val2 inside {[10:15]};};
      $display("After inline constraint: val1 = %0d, val2 = %0d",
item.val1, item.val2);
    end
  end
endmodule
```

2) Inheritance:

```
class parent;
  rand bit [5:0] value;
  constraint value_c {value > 0; value < 10;}
endclass

class child extends parent;
  constraint value_c {value inside {[10:30]};}
endclass

module constraint_inh;
  parent p;
  child c;
  initial begin
    p = new();
    c = new();
    repeat(3) begin
      p.randomize();
      $display("Parent class: value = %0d", p.value);
    end
    repeat(3) begin
      c.randomize();
      $display("Child class: value = %0d", c.value);
    end
  end
endmodule
```

22. Difference between :/ and := operators in randomization

Both are used to assign weightage to different values in the distribution constraints.

:/ Operator

1) For specific value: Assign mentioned weight to that value
2) For range of values *([<range1>: <range2>])*: Assigns weight/(number of value) to each value in that range

:= operator

For a specific value or range of value, the mentioned weight is assigned.

Refer to an example:

```
class seq_item;
  rand bit [7:0] value1;
  rand bit [7:0] value2;

  constraint value1_c {value1 dist {3:/4, [5:8] :/ 7}; }
  constraint value2_c {value2 dist {3:=4, [5:8] := 7}; }

endclass

module constraint_example;
  seq_item item;

  initial begin
    item = new();

    repeat(5) begin
      item.randomize();
      $display("value1 (with :/) = %0d, value2 (with :=)= %0d", item.value1,
item.value2);
    end
  end
endmodule

Output:

value1 (with :/) = 3, value2 (with :=)= 8
value1 (with :/) = 5, value2 (with :=)= 5
value1 (with :/) = 3, value2 (with :=)= 7
value1 (with :/) = 3, value2 (with :=)= 6
value1 (with :/) = 3, value2 (with :=)= 6
```

23. What is std::randomize?

It is one of the methods provided by SystemVerilog to randomize local variable without declaring it as rand or randc. The inline constraint also can be written using the 'with' clause.

Example:

```
int value;

std::randomize(value) with {

                value inside {5, 10, 15, 20};

            };
```

24. Is it possible to call a function from constraint? If yes, explain with an example.

Yes, a function can be called inside a constraint which can take input arguments and also return a value.

Refer to an example:

```
class seq_item;
  rand bit [5:0] value;
  rand bit sel;
  constraint value_c {value == get_values(sel);}

  function bit [5:0] get_values(bit sel);
    return (sel? 'h10: 'h20);
  endfunction
endclass

module constraint_example;
  seq_item item;

  initial begin
    item = new();

    repeat(3) begin
      item.randomize();
      $display("constraint value = %0h", item.value);
    end
    $display("On functiopn call: value = %0h", item.get_values(1));
  end
endmodule
```

25. Write a constraint - divisible by 5.

```
class constraint_example;
  rand bit[3:0] val;
  constraint value_c { val % 5 == 0; }

  function void post_randomize();
    $display("Randomized value = %0d", val);
  endfunction
endclass
```

26. How to disable constraints?

```
class seq_item;
  rand bit [7:0] value1;
  rand bit [7:0] value2;

  constraint value1_c {value1 inside {[10:30]};}
  constraint value2_c {value2 inside {40,70, 80};}

endclass

module constraint_example;
  seq_item item;

  initial begin
    item = new();

    item.randomize();
    $display("Before disabling constraint");
    $display("item: value1 = %0d, value2 = %0d", item.value1, item.value2);

    item.value2_c.constraint_mode(0);  // To disable constraint for value2
using handle item2
    item.randomize();
    $display("After disabling constraint for all value2 alone");
    $display("item: value1 = %0d, value2 = %0d", item.value1, item.value2);
    $display("constraint_mode function returns for value1 = %0d, value2 =
%0d",item.value1_c.constraint_mode(), item.value2_c.constraint_mode());
  end
endmodule
```

27. How to disable randomization?

Yes, it can be disabled using rand_mode. To disable randomization, rand_mode(0) is used. By default, randomization is enabled i.e. rand_mode(1)

Example:

```
class seq_item;
  rand bit [7:0] value1;
  rand bit [7:0] value2;

  constraint value1_c {value1 inside {[10:30]};}
  constraint value2_c {value2 inside {40,70, 80};}

endclass

module constraint_example;
  seq_item item;

  initial begin
    item = new();

    item.randomize();
    $display("Before disabling randomization: value1 = %0d, value2 = %0d",
item.value1, item.value2);

    item.rand_mode(0);  // To disable randomization for all class variables
    item.randomize();
    $display("After disabling randomization for all variables in a class
(Retain old values): value1 = %0d, value2 = %0d", item.value1, item.value2);

    item.rand_mode(1);  // To enable randomization
    item.randomize();
    $display("After enabling randomization: value1 = %0d, value2 = %0d",
item.value1, item.value2);

    item.value2.rand_mode(0);  // To disable randomization for value2
variable alone
    item.randomize();
    $display("After disabling randomization for value2 variables in a class:
value1 = %0d, value2 = %0d", item.value1, item.value2);

    $display("rand_mode function returns for value1 = %0d, value2 = %0d",
item.value1.rand_mode(), item.value2.rand_mode());
  end
endmodule
```

28. Difference between static and dynamic casting

Casting is a process of converting from one data type into another data type for compatibility.

Static Casting	Dynamic Casting
static casting is only applicable to fixed data types.	Dynamic casting is used to cast the assigned values to the variables that might not be ordinarily valid.
It is a compile-time operation	It is a run-time operation
It is a simple and efficient process	It is more complex and less efficient than static casting
Compile failure will be seen for casting failure.	It can detect and handle errors during runtime.

29. Difference between mailbox and queue

A queue is a variable size and ordered collection of elements whereas a mailbox is a communication mechanism that is used to establish the connection between testbench components. One component can put data into a mailbox that stores data internally and can be retrieved by another component. The mailbox can be a parameterized mailbox that can be put or get data of a particular data_type.

The queue has push_front/push_back/pop_front/ pop_back to operate over data elements, whereas a mailbox has get/ put methods as commonly used methods to exchange the data or objects.

30. What is semaphore and in what scenario is it used?

Semaphore is a built-in class in SystemVerilog used for synchronization which is a container that contains a fixed number of keys. It is used to control the access to shared resources.

Example: The same memory location is accessed by two different cores. To avoid unexpected results when cores try to write or read from the same memory location, a semaphore can be used.

Example:

```
module semaphore_example();
  semaphore sem = new(1);

  task write_mem();
    sem.get();
    $display("Before writing into memory");
    #5ns  // Assume 5ns is required to write into mem
    $display("Write completed into memory");
    sem.put();
  endtask

  task read_mem();
    sem.get();
    $display("Before reading from memory");
    #4ns  // Assume 4ns is required to read from mem
    $display("Read completed from memory");
    sem.put();
  endtask

  initial begin
    fork
      write_mem();
      read_mem();
    join
  end
endmodule
```

31. What is input and output skew in clocking block?

To specify the synchronization scheme and timing requirements for an interface, a clocking block is used.

Clocking Skew:

The input or output clocking block signals can be sampled before or after some time unit delay known as clocking skew. It is declared as:

default input #2 output #3;

Where, Input clocking skew: #2 and Output clocking skew: #3

This means input signals is sampled #2 time unit before the clocking event and output signals are driven after #3 time units after the clocking event.

32. What are the types of assertions?

Assertions are used to check design rules or specifications and generate warnings or errors in case of assertion failures.

Types of assertions:

1) Immediate assertions – An assertion that checks a condition **at the current simulation time** is called immediate assertions.
2) Concurrent assertions – An assertion that checks the sequence of events **spread over multiple clock cycles** is called a concurrent assertion.

33. Difference between $strobe, $monitor and $display

System tasks	Description
$display	To display strings, variables, and expressions immediately in the active region.
$monitor	To monitor signal values upon its changes and executes in the postpone region.
$write	To display strings, variables, and expressions without appending the newline at the end of the message and executing in the active region.
$strobe	To display strings, variables, and expressions at the end of the current time slot i.e. in the postpone region.

Refer example to understand more.

```
module display_tb;
  reg [3:0] d1, d2;

  initial begin
    d1 = 4; d2 = 5;

    #5 d1 = 2; d2 = 3;
  end

  initial begin
    $display("At time %0t: {$display A} -> d1 = %0d, d2 = %0d", $time, d1,
d2);
    $monitor("At time %0t: {$monitor A} -> d1 = %0d, d2 = %0d", $time, d1,
d2);
    $write("At time %0t: {$write A} -> d1 = %0d, d2 = %0d", $time, d1, d2);
    $strobe("At time %0t: {$strobe A} -> d1 = %0d, d2 = %0d", $time, d1, d2);
```

```
    #5;

    $display("At time %0t: {$display B} -> d1 = %0d, d2 = %0d", $time, d1,
d2);
    // $monitor is missing -> Observe print for $monitor A
    $write("At time %0t: {$write B} -> d1 = %0d, d2 = %0d", $time, d1, d2);
    $strobe("At time %0t: {$strobe B} -> d1 = %0d, d2 = %0d", $time, d1, d2);
  end
endmodule
```

34. What is ignore bins?

Ignore bins

The ignore bins are used to specify a set of values or transitions that can be excluded from coverage.

Example:

```
module func_coverage;
  logic [3:0] addr;

  covergroup c_group;
    cp1: coverpoint addr {ignore_bins b1 = {1, 10, 12};
                          ignore_bins b2 = {2=>3=>9};
                         }
  endgroup

  c_group cg = new();
  ...
  ...
endmodule
```

35. Difference between ignore and illegal bins.

The ignore bins are used to specify a set of values or transitions that can be excluded from coverage whereas the illegal bins are used to specify a set of values or transitions that can be marked as illegal and a run-time error is reported for the same.

Example:

```
covergroup c_group;
    cp1: coverpoint addr {ignore_bins b1 = {1, 10, 12};
                          ignore_bins b2 = {2=>3=>9};
                         }
    cp2: coverpoint data {illegal_bins b3 = {1, 10, 12};
                          illegal_bins b4 = {2=>3=>9};
                         }
endgroup
```

36. How do you define callback?

SystemVerilog callback

The callbacks are used to alter the behavior of the component without modifying its code. The verification engineer provides a set of hook methods that helps to customize the behavior depending on the requirement. A simple example of callbacks can be the pre_randomize and post_randomize methods before and after the built-in randomize method call.

Callback usage

1) Allows plug-and-play mechanism to establish a reusable verification environment.
2) Based on the hook method call, the user-defined code is executed instead of the empty callback method.

Simply, callbacks are the empty methods that can be implemented in the derived class to tweak the component behavior. These empty methods are called callback methods and calls to these methods are known as callback hooks.

Callback Example

In the below example, modify_pkt is a callback method and it is being called in the pkt_sender task known as callback hook.

The driver class drives a GOOD packet. The err_driver is a derived class of the driver class and it sends a corrupted packet of type BAD_ERR1 or BAD_ERR2 depending on the inject_err bit.

If the inject_err bit is set, then a corrupted packet will be generated otherwise a GOOD packet will be generated.

```
Driver code:

typedef enum {GOOD, BAD_ERR1, BAD_ERR2} pkt_type;

class driver;
  pkt_type pkt;

  task pkt_sender;
    std::randomize(pkt) with {pkt == GOOD;};
    modify_pkt;
  endtask

  virtual task modify_pkt; // callback method
  endtask
endclass
```

```
// Error introduction via err_driver class where callback method modify_pkt
is implemented.
class err_driver extends driver;
  task modify_pkt;
    $display("Injecting error pkt");
    std::randomize(pkt) with {pkt inside {BAD_ERR1, BAD_ERR2};};
  endtask
endclass
```

Environment code:

```
`include "driver.sv"

class env;
  bit inject_err;
  driver drv;
  err_driver drv_err;

  function new();
    drv = new();
    drv_err = new();
  endfunction

  task execute;
    if(inject_err) drv = drv_err;
    // Sending a packet
    drv.pkt_sender();
    $display("Sending packet = %s", drv.pkt.name());
  endtask
endclass
```

Output:

```
Sending packet = GOOD
Sending packet = GOOD
Sending packet = GOOD
Injecting error pkt
Sending packet = BAD_ERR1
Injecting error pkt
Sending packet = BAD_ERR1
Injecting error pkt
Sending packet = BAD_ERR1
Sending packet = GOOD
Sending packet = GOOD
Sending packet = GOOD
```

37. What is DPI? Explain DPI export and import.

SystemVerilog DPI

Direct Programming Interface (DPI) allows users to establish communication between foreign languages and SystemVerilog. It has two separate layers as a foreign language layer and a SystemVerilog layer which are completely isolated. DPI allows having a heterogeneous system that connects and efficiently connects existing code written in other languages.

DPI also allows calling functions and tasks from other languages or vice-versa using import/ export methods. These methods can communicate with the help of arguments and return value.

Import method

SystemVerilog can call functions or tasks (methods) implemented in a foreign language, such methods are called import methods

```
import "DPI-C" function <return_type> <function_name> (<arguments if any>)
```

C file:

```
#include <stdio.h>

  void addition(int a, int b) {
  printf("Addition of %0d and %0d is %0d\n", a, b, a+b);

}
```

SV file:

```
module tb;

  import "DPI-C" function void addition(int a, int b);

  initial
  begin
    $display("Before add function call");
    addition(4,5);
    $display("After add function call");
  end

endmodule
```

Output:

```
Before add function call
Addition of 4 and 5 is 9
After add function call
```

Export method

A foreign language can call functions or tasks (methods) implemented in SystemVerilog, such methods are called export methods

```
export "DPI-C" function <function_name>
```

C file:

```
#include <stdio.h>
//#include <svdpi.h>

extern void addition(int, int);

void c_caller() {
  printf("Calling addition function from c_caller\n");
  addition(4, 5);

}
```

SV file:

```
module tb;

  export "DPI-C" function addition; // This is not a function prototype
  import "DPI-C" context function void c_caller();

  function void addition(int a, b);
    $display("Addition of %0d and %0d is %0d\n", a, b, a+b);
  endfunction

  initial
  begin
    c_caller();
  end
endmodule
```

Output:

```
Calling addition function from c_caller
Addition of 4 and 5 is 9
```

38. What is the implication operator in SVA? Explain its type?

Implication Operator

The implication operator does a property check conditionally if the sequential antecedent is matched.

Syntax:

```
sequence_exp |-> property_exp

sequence_exp |=> property_exp
```

The LHS operand sequence_exp is called an antecedent

The RHS operand property_exp is called a consequent

Type of Implication

1. Overlapped implication
2. Non-overlapped implication

Overlapped implication

The overlapped implication operator is denoted by the |-> symbol.

The evaluation of the consequent starts immediately on the same clock cycle if the antecedent holds true.

The consequent is not evaluated if the antecedent is not true.

Example:

```
property prop;
  @(posedge clk) valid |-> (a ##3 b);
endproperty
```

Non-overlapped implication

The non-overlapped implication operator is denoted by the |=> symbol.

The evaluation of the consequent starts in the next clock cycle if the antecedent holds true.

The consequent is not evaluated if the antecedent is not true.

Example:

```
property prop;
  @(posedge clk) valid |=> (a ##3 b);
endproperty
```

39. What all bins are generated by the following code

```
coverpoint addr {bins b1 = {1, 10, 12};

                 bins b2[] = {[2:9], 11};

                 bins b3[4] = {0:8};
```

variables	bins	Description
addr	bins b1 = {1, 10, 12};	Constructs single bin for 1, 10, 12 value.
	bins b2[] = {[2:9], 11};	Constructs 2 bins ie. b2[0] = 2 ~9, b2[1] = 11.
	bins b3[4] = {0:7};	Constructs 4 bins with possible values as b3[0] = 0 ~1, b3[1] = 1~2, b3[2] = 3~4, b3[3] = 5~7,

Difficult level questions

1. What are the default values of variables in the SystemVerilog classes?

SystemVerilog class has a built-in *new* method which is commonly known as constructor.

Default values for

2 state variables – 0

4 state variables – X

2. What are local and protected access qualifiers?

Local access qualifiers: If a class member is declared as a local, they will be available to that class alone. The child classes will not have access to a local class member of their parent class.

Protected access qualifiers: A protected class member can not be accessed outside class scope except access by their child classes.

3. How do you implement the randc function in SystemVerilog?

```
module tb;
  bit [2:0] data;  // variable which provide random value
  bit [7:0] mask;

  function bit [2:0] my_randc;
    while(1) begin
      data = $random;
      if(!mask[data]) begin
        mask[data] = 1;
        return data;
      end
      else if(&mask) begin
        mask = 0;
        mask[data] = 1;
        break;
      end
    end
    return data;
  endfunction

  initial begin
    repeat(3) begin
      repeat(8)
        $display("data = %0d", my_randc());
```

```
      $display("------------");
    end
  end
endmodule
```

4. Is it possible to generate random numbers without using rand or randc keywords?

Yes, it is possible to generate random numbers using std::randomize() method provided by SystemVerilog to randomize local variable without declaring it as a rand or randc. The inline constraint also can be written using 'with' clause.

Example:

```
int value;
std::randomize(value) with {
                value inside {5, 10, 15, 20};
            };
```

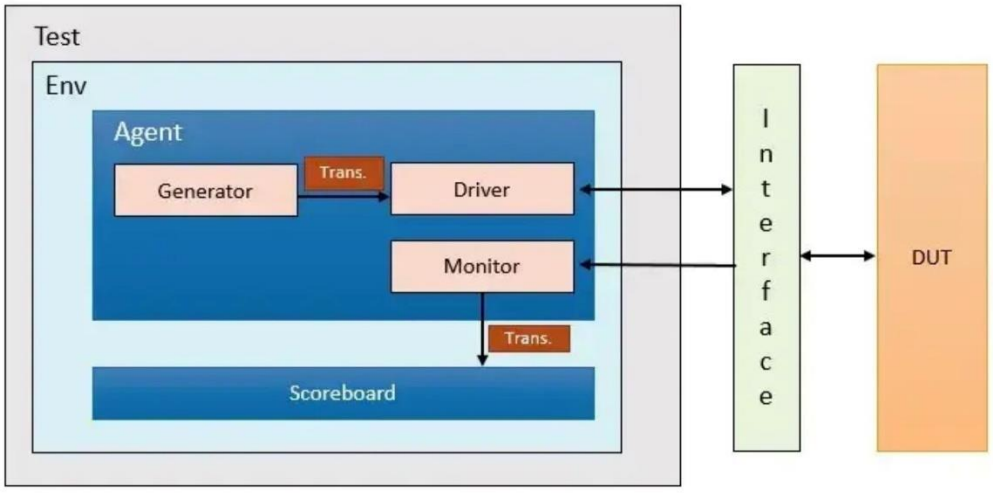
5. Difference between @posedge and $rose?

@(posedge <signal>) is true when its value changes from 0 to 1.

$rose(<signal>) is evaluated to be true for value changes happening across two clocking events from 0 or x or z to 1.

6. Talk about basic testbench components.

Testbench components



Testbench components

1. Transaction
2. Generator
3. Driver
4. Monitor
5. Agent
6. Scoreboard
7. Environment
8. Testbench top
9. Test

Transaction

The transaction is a packet that is driven to the DUT or monitored by the monitor as a pin-level activity.

In simple terms, the transaction is a class that holds a structure that is used to communicate with DUT.

Generator

The generator creates or generates randomized transactions or stimuli and passes them to the driver.

Driver

The driver interacts with DUT. It receives randomized transactions from the generator and drives them to the driven as a pin level activity.

Monitor

The monitor observes pin-level activity on the connected interface at the input and output of the design. This pin-level activity is converted into a transaction packet and sent to the scoreboard for checking purposes.

Agent

An agent is a container that holds the generator, driver, and monitor. This is helpful to have a structured hierarchy based on the protocol or interface requirement.

Scoreboard

The scoreboard receives the transaction packet from the monitor and compares it with the reference model. The reference module is written based on design specification understanding and design behavior.

Environment

An environment allows a well-mannered hierarchy and container for agents, scoreboards.

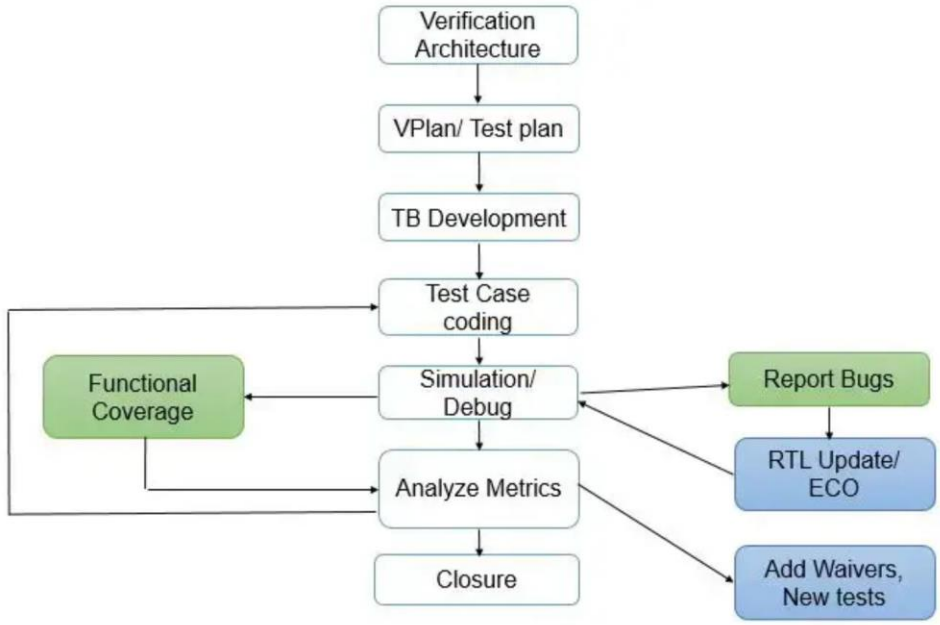
Testbench top

The testbench top is a top-level component that includes interface and DUT instances. It connects design with the testbench.

Test

The test is at the top of the hierarchy that initiates the environment component construction and connection between them. It is also responsible for the testbench configuration and stimulus generation process.

7. Explain the cycle of verification and its closure.



ASIC Verification Flow

Verification Architecture

In the verification architectural phase, engineers decide what all verification components are required.

Verification Plan/ Testplan

The verification plan includes a test plan(list of test cases that target design features), functional coverage planning, module/block assignments to the verification engineers, checker, and assertion planning. The verification plan also involves planning for how verification components can be reused at system/ SOC level verification.

Testbench Development

As a part of testbench development, verification engineers develop testbench components, interface connections with the DUT, VIP integration with a testbench, inter-component connections within testbench (like monitor to scoreboard connection), etc.

Testcase Coding

A constraint-based random or dedicated test case is written for single or multiple features in the design. A test case also kicks off UVM-based sequences to generate required scenarios.

Simulation/ Debug

In this phase, engineers validate whether a specific feature is targetted or not, If not, again test case is modified to target the feature. With the help of a checker/ scoreboard, the error is reported if the desired design does not behave as expected. Using waveform analysis or log prints, the design or verification environment is judged and a bug is reported to the design team if it comes out to be a design issue otherwise, simulation is re-run after fixing the verification component.

Analyze Metrics

Assertions, code, and functional coverage are common metrics that are used as analysis metrics before we close the verification of the design.

8. How will you test the functionality of interrupts using functional coverage?

The functionality of interrupt getting raised and being serviced by the testbench can be verified by writing 'Sequence of transitions' coverpoint as follows

```
covergroup c_group;
    cp1: coverpoint intr {bins b1 = (0 => 1 => 0);
                          }
Endgroup
```

This allows testing interrupt is generated due to the stimulus and testbench is calling an appropriate ISR to service the interrupt.

9. What is layered architecture in Verification?

Layered architecture involves structuring the verification environment into various layers or levels that help to provide abstraction, scalability, reusability, etc.

Testbench Top and Test Layer: The testbench top is a top-level component that includes interface and DUT instances. It connects the design with the test bench. The reset, clock generation, and its connection with DUT is also done in testbench top.

The test is at the top of the hierarchy that initiates the environment component construction and connection between them. It is also responsible for the testbench configuration and stimulus generation process.

Based on the design feature verification, a directed or constrained random test is written. The test case generates stimulus based on configurations or

Verification components Layer:

The Verification Components Layer serves as an abstraction that encapsulates the behavior of the Design Under Test (DUT) while focusing on verifying the functionality of the design using various verification components.

Coverage Layer: Coverage collection mechanisms track different aspects of the DUT that have been exercised during simulation. This includes functional coverage, code coverage, and assertion coverage, providing insights into the verification completeness and identifying any areas that require additional testing.

Advantages of the layered approach:

1. Improved Verification Efficiency: Abstraction and modularity accelerate the verification process.
2. Enhanced Reusability: Verification components can be reused across multiple projects.
3. Better Test Coverage: A modular approach facilitates comprehensive testing.
4. Improved Maintainability: Focused components make verification code easier to maintain and update.

10. How can you establish communication between monitor and scoreboard in SystemVerilog?

The monitor observes pin-level activity on the connected interface at the input and output of the design. This pin-level activity is converted into a transaction packet and sent to the scoreboard for checking purposes.

The scoreboard receives the transaction packet from the monitor and compares it with the reference model. The reference module is written based on design specification understanding and design behavior.

Both are used to verify the correctness of the design in the verification environment component. They are connected using a mailbox in a SystemVerilog-based verification environment.

11. Difference between class-based testbench and module-based testbench.

class-based testbench	module-based testbench
Testbench is developed around multiple blocks of connected design modules	Testbench is developed around individual design modules.
The class-based stimulus is generated and driven to the design.	The physical interface is driven to the design
Provide more flexibility in terms of configuring testbench in various modes as needed.	Configuring a testbench needs modification in the parameter used for the corresponding module that does not give the flexibility to change it during run time.
Useful for complex design verifications	Useful for smaller and simpler design verification.

12. How will be your approach if code coverage is 100% but functional coverage is too low?

This can be possible when

1. Functional coverage bins (may be auto bins) generated for wide variable range which is not supported by design.
2. Cross check if code coverage exclusions are valid so that it should not give false interpretation of 100% code coverage.
3. Functional coverage implemented for feature (were planned during initial project phasing), but those are not supported by the design now.

13. How will be your approach if functional coverage is 100% but code coverage is too low?

This can be possible when

1. RTL code is not covered, so new stimulus or existing stimulus improvement is required (assuming dead code is not a reason for low code coverage).
2. Another part is that 100% functional coverage suspects that covergroup for some features might be missing. A detailed analysis is required to root cause of the same. If the functional coverage is properly implemented, then code coverage can be improved by new stimulus addition, and updating existing stimulus which might include constraint-based testing.

14. Write an assertion for glitch detection.

```
realtime duration=50ns;
 property glitch_detection;
    realtime first_change;
   @(signal)  // Change in signal value (posedge and negedge)
     (1, first_change = $realtime) |=> (($realtime - first_change) >=
duration); // It saves current time and also check the delay from previous
edge
  endproperty
  ap_glitch_p: assert property(glitch_detection);
```